SQL Migration Runbook

When you migrate versions of SQL it's best practice to follow a specific set of steps (Runbook) to make sure you check and prep items before, during and after the migration to make sure the migration goes well and you run into little or no issues.

Below is a general outline of the migration process that works for most in-place migrations of SQL server.

It's best practice to perform the migration in a Dev/Test environment first to discover and correct any issues you may find before moving to a Production environment. This will help keep things smooth and help pin point the issues and nuances that may occur with the migration as in a Production environment, speed and stability are key. It's also a good idea to do the migration in 2 phases, first is the SQL Server Engine migration (the actual install of SQL server), and second is the SQL Optimizer & Query Processor migration (called Compatibility Level) for each user database.

Phase 1 - SQL Server Engine Migration

ROLLBACK PLAN (IF NEEDED) - For this phase you will need to recover the VM level snapshot/backup you created below as the install modifies system and application files that will need to be reverted back to the previous versions.

Step 1.1 - General Preparation Checklist

- □ Obtain the SQL server image for the version and edition you are wanting to migrate to.
- □ Download the latest Cumulative Update for the desired SQL server version.
- □ Download latest version of SSMS (if needed) as some versions of SQL can behave differently with older versions of SSMS if you use that to manage your SQL server.
- □ Make sure you either obtain a proper SQL license, if you're in Azure and use the Pay As You Go license model, Microsoft provides the activation for you.
- □ Make sure to have a way to make a VM level snapshot/backup as close to the deployment window as possible, this way you can recover if the install is interrupted, corrupted or the migration just simply doesn't work for you environment.
- □ Setup Smoke Testing, query function testing with appropriate teams following migration phases.

You can go to this website to find links to all SQL server images and installers, cumulative updates, and SSMS versions
https://www.mssqltips.com/sqlservertip/6429/sql-server-download-quick-links/

Step 1.2 - Pre-Migration Checks

These are some checks to perform before you do the migration, save these results to a file so you have them for reference. This will give you a general overview of what you currently have as a base for comparison later if needed.

If needed, you can install the latest version of SSMS on the targeted server. Install only SSMS (not SSIS, SSRS, etc.) as this could break other SQL services that are on that server. You can re-run the installer to migrate those at a later time if desired.

Check current SQL version

```
SELECT @@SERVERNAME AS server_name, @@VERSION AS version_string;
SELECT SERVERPROPERTY('ProductVersion') AS product_version,
    SERVERPROPERTY('ProductLevel')   AS product_level,
    SERVERPROPERTY('Edition')       AS edition,
    SERVERPROPERTY('EngineEdition')  AS engine_edition;
```

Database List (including state, compatibility and recovery mode)

```
SELECT name, state_desc, recovery_model_desc, compatibility_level
FROM sys.databases
ORDER BY name;
```

Core Server Configuration Settings

```
EXEC sp_configure;
```

SQL Server Job List and Last Run Status

```
;WITH j AS (
  SELECT j.job_id, j.name
  FROM msdb.dbo.sysjobs j
)
SELECT TOP (200)
  j.name AS job_name,
  h.run_date, h.run_time, h.run_duration,
  h.step_id, h.sql_severity, h.message,
  CASE h.run_status
    WHEN 0 THEN 'Failed'
    WHEN 1 THEN 'Succeeded'
    WHEN 2 THEN 'Retry'
    WHEN 3 THEN 'Canceled'
    WHEN 4 THEN 'In progress'
  END AS run_status
FROM j
JOIN msdb.dbo.sysjobhistory h ON j.job_id = h.job_id
WHERE h.step_id = 0
ORDER BY h.instance_id DESC;
```

Query Store Settings

```
SELECT name, is_query_store_on
FROM sys.databases
ORDER BY name;
```

Step 1.3 - Replication Checks (If needed - Skip if you are not using Replication)

If you use Replication on the server you are migrating you will want to run checks on the configurations for that as well.

List Databases That Are Being Replicated

```
SELECT name, is_published, is_subscribed
FROM sys.databases
WHERE is_published = 1 OR is_subscribed = 1;
```

Replication Job Inventory (shows replication jobs, last run, last status, etc.)

```
;WITH ReplicationJobs AS
(
   SELECT
      j.job_id,
      j.name AS JobName,
      c.name AS CategoryName,
      j.enabled
   FROM msdb.dbo.sysjobs j
   LEFT JOIN msdb.dbo.syscategories c
      ON j.category_id = c.category_id
   WHERE c.name LIKE 'REPL-%'
     OR j.name LIKE '%Log Reader%'
     OR j.name LIKE '%Snapshot%'
     OR j.name LIKE '%Distribution%'
     OR j.name LIKE '%Merge%'
     OR j.name LIKE '%Queue Reader%'
),
LastJobHistory AS
```

```
(
  SELECT
    h.job_id,
    h.instance_id,
    h.step_id,
    h.run_status,
    h.run_date,
    h.run_time,
    h.run_duration,
    h.message,
    ROW_NUMBER() OVER
    (
      PARTITION BY h.job_id
      ORDER BY h.instance_id DESC
    ) AS rn
  FROM msdb.dbo.sysjobhistory h
  WHERE h.step_id = 0
)
SELECT
  rj.JobName,
  rj.CategoryName,
  rj.enabled,
  CASE ljh.run_status
    WHEN 0 THEN 'Failed'
    WHEN 1 THEN 'Succeeded'
    WHEN 2 THEN 'Retry'
    WHEN 3 THEN 'Canceled'
    WHEN 4 THEN 'In Progress'
    ELSE 'Unknown'
  END AS LastRunStatus,
  msdb.dbo.agent_datetime(ljh.run_date, ljh.run_time) AS LastRunDateTime,
  STUFF(STUFF(RIGHT('000000' + CAST(ljh.run_duration AS varchar(6)), 6), 3, 0, ':'), 6, 0, ':') AS RunDuration,
  ljh.message AS LastMessage
FROM ReplicationJobs rj
LEFT JOIN LastJobHistory ljh
  ON rj.job_id = ljh.job_id
 AND ljh.rn = 1
ORDER BY rj.CategoryName, rj.JobName;
```

Step 1.4 - Perform Migration

- Mount SQL Server Image and run setup.exe, or if using an installer run the executable.
- Follow all the prompts, when prompted for a Product Key enter it as needed, if you are using Azure Pay as You Go do not enter a key, select a free version or evaluation and proceed as Microsoft has a service that synchronizes the activation after install.
- Make sure all install locations and instances look correct during the installation.
- Once installation is complete, restart the entire server computer.
- Once the computer is back up, run the following query to confirm you are on the desired SQL version

```
SELECT @@SERVERNAME AS server_name, @@VERSION AS version_string;
SELECT SERVERPROPERTY('ProductVersion') AS product_version,
    SERVERPROPERTY('ProductLevel')   AS product_level,
    SERVERPROPERTY('Edition')        AS edition,
    SERVERPROPERTY('EngineEdition')  AS engine_edition;
```

- If correct, proceed to run the latest Cumulative Update installer.
- Reboot Server computer after install.
- Re-run above command to confirm Cumulative Update is installed.

Step 1.5 - Immediate Post Migration Checks

Re-run all the Pre-Migration checks listed in Steps 1.2 and 1.3 to make sure everything is enabled and configured correctly. You will want to check the two things below as well.

Make sure the SQL Server and SQL Server Agent services are running on the computer

Check Error Logs

```
DECLARE @StartDate DATETIME = DATEADD(HOUR, -24, GETDATE());
DECLARE @EndDate   DATETIME = GETDATE();

EXEC xp_readerrorlog
  0,
  1,
  NULL,
  NULL,
  @StartDate,
  @EndDate,
  N'desc';
```

If everything looks fine, have testing group proceed with their tests. Keep checking the error logs to make sure nothing shows up, you can also monitor query activity during testing using the query below.

Check Current Running Queries

```
SELECT
  s.session_id,
  s.login_name,
  s.host_name,
  s.program_name,
  r.status,
  r.command,
  r.start_time,
  r.total_elapsed_time,
  r.reads,
  r.writes,
  DB_NAME(r.database_id) AS database_name,
  SUBSTRING(st.text, (r.statement_start_offset / 2) + 1,
        ((CASE r.statement_end_offset
          WHEN -1 THEN DATALENGTH(st.text)
          ELSE r.statement_end_offset
         END - r.statement_start_offset) / 2) + 1) AS current_statement_text
FROM sys.dm_exec_sessions AS s
INNER JOIN sys.dm_exec_requests AS r ON s.session_id = r.session_id
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) AS st
WHERE s.is_user_process = 1;
```

After Smoke/Post-Migration testing is completed and signed off on. Continue to lightly monitor for several days (3-4) after to confirm everything is operating correctly. If no issues are reported, you can now move to Phase 2 of the SQL migration.

Phase 2 - SQL Optimizer and Query Processor Migration

ROLLBACK PLAN (IF NEEDED) - For this phase you can revert the Compatibility Level for all or specific databases, instructions are in Step 2.4. For a database recovery, you only really need to do this in rare cases where the entire database is either corrupted or not working at all but are nice to have just in case.

IMPORTANT NOTE - Don't change the compatibility level on msdb, master, model, tempdb, distribution (aka Replication), ReportServer and ReportServerTempDB (aka SSRS) as changing the level isn't usually needed, has no benefit and can cause instability points.

Step 2.1 - General Preparation Checklist

- □ Have database level backups in place either Full (preferrable), or Full + Differentials (if you have a backup system in place already) done as close to the migration as possible just in case a rollback is needed.
- □ Setup another Smoke and query function testing session with your testing group.
- □ Make list of user databases you want to adjust the Compatibility Level on.

Step 2.2a - Pre-Migration Checks

You'll want to do a few pre migration checks before adjusting the compatibility levels on the databases to be used as a configuration backup and a baseline to compare with throughout this phase of the migration. These can be done multiple times before the compatibility level migration if desired to get a more wider scope of query analysis for post-migration comparisons.

Database List (including state, compatibility and recovery mode)

```
SELECT name, state_desc, recovery_model_desc, compatibility_level
FROM sys.databases
ORDER BY name;
```

Query Store Health Check (checks current size, state, etc.)

Check the databases you are wanting to migrate and make sure the Query Store is enabled and in READ/WRITE mode, if they are not enabled, skip to Step 2.2b temporarily before running any more checks.

```
IF OBJECT_ID('tempdb..#QueryStoreStatus') IS NOT NULL
   DROP TABLE #QueryStoreStatus;

CREATE TABLE #QueryStoreStatus
(
   database_name SYSNAME,
   compatibility_level INT,
   actual_state_desc NVARCHAR(60),
   desired_state_desc NVARCHAR(60),
   readonly_reason BIGINT,
   readonly_reason_desc NVARCHAR(200),
   current_storage_size_mb FLOAT,
   max_storage_size_mb FLOAT,
   percent_used DECIMAL(9,2)
);

DECLARE @db SYSNAME;
DECLARE @compat INT;
DECLARE @sql NVARCHAR(MAX);

DECLARE dbs CURSOR LOCAL FAST_FORWARD FOR
SELECT name, compatibility_level
FROM master.sys.databases
WHERE database_id > 4
  AND state_desc = N'ONLINE';

OPEN dbs;
FETCH NEXT FROM dbs INTO @db, @compat;

WHILE @@FETCH_STATUS = 0
BEGIN
   SET @sql = N'
   USE ' + QUOTENAME(@db) + N';

   INSERT INTO #QueryStoreStatus
   (
      database_name,
      compatibility_level,
```

```
    actual_state_desc,
    desired_state_desc,
    readonly_reason,
    readonly_reason_desc,
    current_storage_size_mb,
    max_storage_size_mb,
    percent_used
  )
  SELECT
    DB_NAME(),
    @compatibility_level,
    q.actual_state_desc,
    q.desired_state_desc,
    q.readonly_reason,
    CASE
      WHEN q.readonly_reason = 0 THEN ''NONE''
      WHEN (q.readonly_reason & 1) = 1 THEN ''DATABASE_READ_ONLY''
      WHEN (q.readonly_reason & 2) = 2 THEN ''SINGLE_USER''
      WHEN (q.readonly_reason & 4) = 4 THEN ''EMERGENCY''
      WHEN (q.readonly_reason & 8) = 8 THEN ''SECONDARY_REPLICA''
      WHEN (q.readonly_reason & 65536) = 65536 THEN ''SIZE_LIMIT_REACHED''
      ELSE ''OTHER / COMBINATION''
    END,
    q.current_storage_size_mb,
    q.max_storage_size_mb,
    CASE
      WHEN q.max_storage_size_mb = 0 THEN NULL
      ELSE CAST((q.current_storage_size_mb * 100.0) / q.max_storage_size_mb AS DECIMAL(9,2))
    END
  FROM sys.database_query_store_options AS q;';

  EXEC sys.sp_executesql
    @sql,
    N'@compatibility_level INT',
    @compatibility_level = @compat;

  FETCH NEXT FROM dbs INTO @db, @compat;
END

CLOSE dbs;
DEALLOCATE dbs;

SELECT
  database_name,
  compatibility_level,
  actual_state_desc,
  desired_state_desc,
  readonly_reason,
  readonly_reason_desc,
  current_storage_size_mb,
  max_storage_size_mb,
  percent_used
FROM #QueryStoreStatus
ORDER BY
  CASE WHEN actual_state_desc <> N'READ_WRITE' THEN 0 ELSE 1 END,
  percent_used DESC,
  database_name;
```

<u>*The below query will need to be run against each database you wish to migrate.***</u>**

Top 50 Queries Before/After Combability Change

Set the @ChangeTime to the date/time of the combability change, change/add databases to @Databases for the databases you changed the level too. With the results, use the chart below for comparisons…

Case 1 — Clear regression

Duration ↑
CPU ↑
Reads ↑

Summary - Something changed (likely plan)

Case 2 — Suspicious but unclear

Duration ↑
CPU same
Reads same

Summary - Could be:

- blocking
- waits
- low sample size

Case 3 — Ignore

Duration ↑ slightly
CPU same
Reads same

Summary - Noise

Case 4 — Improvement

Duration ↓
CPU ↓ or same
Reads ↓ or same

Summary - Good change

```
SET NOCOUNT ON;

USE master;

DECLARE @ChangeTime datetime2 = '2026-04-25 20:00:00';

DECLARE @Databases TABLE (database_name sysname);

INSERT INTO @Databases
VALUES
   ('Database1'),
   ('Database2'),
   ('Database3'),
   ('Database4'),
   ('Database5');

DROP TABLE IF EXISTS #QueryStoreCompare;

CREATE TABLE #QueryStoreCompare
(
```

```
    database_name sysname,
    query_id bigint,
    query_sql_text nvarchar(max),
    comparison_period varchar(10),
    total_executions bigint,
    avg_duration_ms decimal(18,2),
    avg_cpu_ms decimal(18,2),
    avg_logical_reads decimal(18,2)
);

DECLARE @DatabaseName sysname;
DECLARE @sql nvarchar(max);

DECLARE db_cursor CURSOR LOCAL FAST_FORWARD FOR
SELECT database_name FROM @Databases;

OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @DatabaseName;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @sql = N'
    USE ' + QUOTENAME(@DatabaseName) + N';

    INSERT INTO #QueryStoreCompare
    SELECT
        DB_NAME(),
        q.query_id,
        qt.query_sql_text,
        CASE WHEN rsi.start_time < @ChangeTime THEN ''Before'' ELSE ''After'' END,
        SUM(rs.count_executions),

        CAST(
            SUM(rs.avg_duration * rs.count_executions)
            / NULLIF(SUM(rs.count_executions), 0) / 1000.0
            AS decimal(18,2)
        ) AS avg_duration_ms,

        CAST(
            SUM(rs.avg_cpu_time * rs.count_executions)
            / NULLIF(SUM(rs.count_executions), 0) / 1000.0
            AS decimal(18,2)
        ) AS avg_cpu_ms,

        CAST(
            SUM(rs.avg_logical_io_reads * rs.count_executions)
            / NULLIF(SUM(rs.count_executions), 0)
            AS decimal(18,2)
        ) AS avg_logical_reads

    FROM sys.query_store_query q
    JOIN sys.query_store_query_text qt
        ON q.query_text_id = qt.query_text_id
    JOIN sys.query_store_plan p
        ON q.query_id = p.query_id
    JOIN sys.query_store_runtime_stats rs
        ON p.plan_id = rs.plan_id
    JOIN sys.query_store_runtime_stats_interval rsi
        ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
    WHERE rsi.start_time >= DATEADD(day, -14, @ChangeTime)
      AND rsi.start_time <  DATEADD(day,  14, @ChangeTime)
```

```
    GROUP BY
        q.query_id,
        qt.query_sql_text,
        CASE WHEN rsi.start_time < @ChangeTime THEN ''Before'' ELSE ''After'' END;
    ';

    EXEC sys.sp_executesql
        @sql,
        N'@ChangeTime datetime2',
        @ChangeTime = @ChangeTime;

    FETCH NEXT FROM db_cursor INTO @DatabaseName;
END

CLOSE db_cursor;
DEALLOCATE db_cursor;

SELECT
    database_name,
    query_id,
    LEFT(MAX(query_sql_text), 4000) AS query_sql_text,

    SUM(CASE WHEN comparison_period = 'Before' THEN total_executions ELSE 0 END) AS before_executions,
    SUM(CASE WHEN comparison_period = 'After'  THEN total_executions ELSE 0 END) AS after_executions,

    MAX(CASE WHEN comparison_period = 'Before' THEN avg_duration_ms END) AS before_avg_duration_ms,
    MAX(CASE WHEN comparison_period = 'After'  THEN avg_duration_ms END) AS after_avg_duration_ms,

    MAX(CASE WHEN comparison_period = 'Before' THEN avg_cpu_ms END) AS before_avg_cpu_ms,
    MAX(CASE WHEN comparison_period = 'After'  THEN avg_cpu_ms END) AS after_avg_cpu_ms,

    MAX(CASE WHEN comparison_period = 'Before' THEN avg_logical_reads END) AS before_avg_logical_reads,
    MAX(CASE WHEN comparison_period = 'After'  THEN avg_logical_reads END) AS after_avg_logical_reads

FROM #QueryStoreCompare
GROUP BY
    database_name,
    query_id
HAVING
    SUM(CASE WHEN comparison_period = 'Before' THEN total_executions ELSE 0 END) > 0
AND SUM(CASE WHEN comparison_period = 'After'  THEN total_executions ELSE 0 END) > 0
ORDER BY
    database_name,
    after_avg_duration_ms DESC;
```

Step 2.2b - Enable The Query Store (If needed)

Before you can take a proper baseline for the compatibility level change you will want to enable the query store on all the databases you are wanting to adjust the level on. Microsoft recommends this for these changes, and it's not a bad thing to have enabled in general for any type of query investigation and troubleshooting you need to do. This can be enabled at any time but being it does alter the database it is best to run this after hours or during a maintenance window.

After running the script, re-run the "**Query Store Health Check**" from Step 2.2a again to confirm all databases are configured properly. If so, continue your checks in Step 2.2a.

Enable Query Store

Replace databases under "**INSERT INTO @Databases**" with the Databases you want to modify

```
    DECLARE @Databases TABLE (DatabaseName sysname);

    INSERT INTO @Databases (DatabaseName)
```

```
VALUES
  ('DB1'),
  ('DB2'),
  ('DB3');  -- add more as needed

DECLARE @DBName sysname;
DECLARE @SQL nvarchar(max);

DECLARE db_cursor CURSOR FOR
SELECT DatabaseName FROM @Databases;

OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @DBName;

WHILE @@FETCH_STATUS = 0
BEGIN
  SET @SQL = N'
  ALTER DATABASE [' + @DBName + N'] SET QUERY_STORE = ON;

  ALTER DATABASE [' + @DBName + N']
  SET QUERY_STORE (
    OPERATION_MODE = READ_WRITE,
    QUERY_CAPTURE_MODE = AUTO,
    MAX_STORAGE_SIZE_MB = 2048,
    SIZE_BASED_CLEANUP_MODE = AUTO,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 14),
    INTERVAL_LENGTH_MINUTES = 60,
    DATA_FLUSH_INTERVAL_SECONDS = 900
  );';

  PRINT 'Processing: ' + @DBName;
  EXEC sp_executesql @SQL;

  FETCH NEXT FROM db_cursor INTO @DBName;
END

CLOSE db_cursor;
DEALLOCATE db_cursor;
```

Step 2.3 - Adjust Compatibility Level

Once you're ready to make the change and ran all baselines and made sure backups occurred. You can run the following query to change the compatibility level. Even though it is possible to change it all once, it's best to change it one at a time to make sure you can isolate issues properly. Adjust database name and COMPATIBILITY_LEVEL for your environment.

A list of all the SQL Server compatibility levels can be found at this website.
https://learn.microsoft.com/en-us/sql/t-sql/statements/alter-database-transact-sql-compatibility-level?view=sql-server-ver17

```
ALTER DATABASE YourDB SET COMPATIBILITY_LEVEL = 160;
```

Step 2.4 - Immediate Post Migration Checks

Have the testers begin testing and while they are performing those test, run all the queries from Step 2.2a. The main things to check are making sure the compatibility level did change for your targeted databases, and to compare your current query store results against your baseline to see if you notice and major increases in the values. You will probably need to run these a few times to get a more broad comparative analysis.

After testing is complete and signed off, continue to check the query stores periodically for several days (3-4) and watch for any large changes in plans. If you hear any issues or see any problems make note of the plan, it may need to change to a forced plan. If there is catastrophic issues, or if the decision is made to revert the compatibility back to what it was before, you just follow rollback and use the

same command in Step 2.3 but with the older compatibility level. If not critical, it's recommended to run that command after hours or during a maintenance window as it does lock the database when the command is run.

If there are no issues and your post checks all come back good your SQL migration is fully completed.